

CS437/537 - Quiz Benzeri Deneme Sınavı

Kaynak: quiz-2-key.pdf, quiz-3-key.pdf, quiz-4-key.pdf, quiz-5-key.pdf, quiz-6-key.pdf, quiz-7-key.pdf, quiz-8-key.pdf

Bu PDF, yüklediğiniz quiz anahtarlarının soru tiplerine göre hazırlanmıştır. Sorular özellikle aynı mantığı ölçer; birkaç yerde sınav pratiği için bir tık daha zorlaştırılmıştır. Cevap anahtarı ikinci bölümde verilmiştir.

Bölüm 1 - Sorular

Soru 1 - LOC, LLOC, Halstead, CC, nesting ve clone türü

Aşağıdaki Java kodunu inceleyin.

```
public class InvoiceAnalyzer {
    public int calculateTotal(int[] prices) {
        int total = 0;
        for (int i = 0; i < prices.length; i++) {
            if (prices[i] > 100) {
                total = total + prices[i] - 10;
            } else {
                total = total + prices[i];
            }
        }
        return total;
    }

    public int calculateSum(int[] values) {
        int result = 0;
        for (int j = 0; j < values.length; j++) {
            result = result + values[j];
        }
        return result;
    }
}
```

- Q1.1 Physical LOC kaçtır? Boş satırları saymayın, süslü parantezleri sayın.
- Q1.2 Logical LOC kaçtır? Sadece gerçek ifadeleri/statements sayın; class/method imzalarını, braces ve boş satırları saymayın.
- Q1.3 calculateTotal metodu için Cyclomatic Complexity değerini karar noktası + 1 kuralıyla hesaplayın.
- Q1.4 calculateTotal metodu için maksimum nesting seviyesi kaçtır?
- Q1.5 calculateTotal ve calculateSum arasında clone türü var mı? Varsa hangi tür? Kısa açıklayın.
- Q1.6 calculateSum metodunun Halstead değerleri için şu varsayımı kullanın: Operatörler = {int, =, for, <, ++, +, [], return}. Operandlar = {result, j, values, length, 0}. N1=14, N2=16 veriliyor. n1, n2, N, n ve V değerlerini hesaplayın. $V = N * \log_2(n)$.

Soru 2 - CK OO metrikleri: WMC, DIT, NOC, CBO, RFC, LCOM

Aşağıdaki sınıf ve hiyerarşileri inceleyin.

```
class Account {
    protected double balance;
    public void deposit(double amount) { balance += amount; }
    public void withdraw(double amount) {
        if (amount <= balance) balance -= amount;
    }
}

class SavingsAccount extends Account {
    public void addInterest() { balance = balance * 1.05; }
}

class PremiumSavingsAccount extends SavingsAccount {
    public void applyCashback(double purchase) {
        if (purchase > 1000) balance += purchase * 0.02;
        else balance += purchase * 0.01;
    }
}

class LoanAccount extends Account {
    public void calculatePenalty(int daysLate) {
        if (daysLate > 30) System.out.println("High penalty");
    }
}
```

```

        else System.out.println("Low penalty");
    }
}

class CustomerProfile {
    private String name;
    private String email;
    private Account account;
    private EmailSender sender;

    public void updateName(String n) { name = n; }
    public void updateEmail(String e) { email = e; }
    public void sendWelcomeMail() { sender.send(email); }
    public void printBalance() { System.out.println(account.balance); }
}

```

- Q2.1 Verilen karmaşıklık puanlarını kullanarak WMC(PremiumSavingsAccount) hesaplayın: applyCashback karmaşıklık 2; diğer tüm basit metotlar 1. WMC sadece sınıfın kendi metotları için hesaplınsın.
- Q2.2 DIT(PremiumSavingsAccount) kaçtır? Object hariç, Account kök sayılacak şekilde hesaplayın.
- Q2.3 NOC(Account) kaçtır?
- Q2.4 CustomerProfile için CBO kaçtır? Sadece doğrudan kullandığı diğer sınıfları sayın.
- Q2.5 CustomerProfile için RFC kaçtır? Sınıfın kendi metotlarını ve doğrudan çağrılan metotları benzersiz sayın.
- Q2.6 CustomerProfile için LCOM hesaplayın. Metot çiftleri üzerinden P =ortak alan kullanan çift sayısı, Q =ortak alan kullanmayan çift sayısı. $LCOM = \max(Q-P, 0)$.

Soru 3 - Code smells ve refactoring

Aşağıdaki sınıfı code smell açısından analiz edin ve refactor önerin.

```

class SubscriptionManager {
    private List<User> users = new ArrayList<>();

    public void addUser(User u) { users.add(u); }

    public void chargeUsers(String planType) {
        for (User u : users) {
            double price = 0;
            switch(planType) {
                case "BASIC": price = 9.99; break;
                case "PRO": price = 19.99; break;
                case "ENTERPRISE": price = 49.99; break;
                default: price = 0;
            }
            if (u.isStudent()) price = price * 0.80;
            System.out.println("Charging " + u.getEmail() + ": " + price);
        }
    }

    public void exportUsersToCsv(String file) {
        // open file, write user email and plan, close file
    }

    public void sendRenewalEmails() {
        for (User u : users) {
            System.out.println("Sending email to " + u.getEmail());
        }
    }
}

```

- Q3.1 En az 5 code smell belirleyin.
- Q3.2 Hangi refactoringleri uygularsınız? En az 4 madde yazın.
- Q3.3 Switch/type conditional kokusunu gidermek için hangi pattern daha uygundur? Kısa sınıf taslağı verin.
- Q3.4 Refactoring sonrası SRP ve OCP nasıl iyileşir?

Soru 4 - SOLID ihlali analizi ve düzeltme

Aşağıdaki senaryoda mevcut tasarım monolitikdir. Yeni talepler geliyor.

```

class DeliveryOrderProcessor {
    public void process(Order order) {
        double total = order.calculateTotal();

        if (order.customer().isGold()) total *= 0.85;

        if (order.paymentType().equals("CARD")) {
            new CardPayment().pay(total);
        }
    }
}

```

```

        } else {
            new CashPayment().pay(total);
        }

        new MySQLOrderRepository().save(order);
        new EmailNotifier().send(order.customer().email(), "Order accepted");
    }
}

abstract class Courier {
    abstract void deliver(Order order);
}

class DroneCourier extends Courier {
    void deliver(Order order) {
        if (order.weight() > 5) throw new UnsupportedOperationException();
        System.out.println("Drone delivery");
    }
}

interface CustomerLogin {
    void loginWithPassword(String username, String password);
    void loginWithOTP(String phone);
    void loginWithFaceId();
}

class PasswordOnlyLogin implements CustomerLogin {
    public void loginWithPassword(String u, String p) { }
    public void loginWithOTP(String phone) { throw new UnsupportedOperationException(); }
    public void loginWithFaceId() { throw new UnsupportedOperationException(); }
}

```

- Q4.1 Hangi SOLID prensipleri ihlal ediliyor? Her ihlal için koddan somut örnek gösterin.
- Q4.2 Yeni feature: Apple Pay, SMS bildirimi, dinamik kampanya indirimleri ve MongoDB desteği eklenecek. Her feature için hangi abstraction/interface gerekir?
- Q4.3 DroneCourier tasarımındaki LSP problemini nasıl düzeltirsiniz?
- Q4.4 CustomerLogin arayüzündeki ISP problemini nasıl düzeltirsiniz?
- Q4.5 Refactor edilmiş DeliveryOrderProcessor constructor bağımlılıklarını yazın.

Soru 5 - Creational GoF pattern seçimi ve entegre tasarım

Bir online eğitim platformunda sınav oluşturma sistemi tasarlanacaktır. Sistem; çoktan seçmeli, kodlama ve açık uçlu soru tipleri üretmelidir. Platform ayrıca Üniversite, Kurumsal ve Bootcamp gibi farklı sınav aileleri destekler. Her aile; soru, değerlendirme politikası ve rapor formatı açısından uyumlu nesneler üretmelidir. Bazı sınav şablonları pahalı hazırlandığı için klonlanabilmelidir. Sınav raporu çok sayıda opsiyonel bölümle adım adım oluşturulmalıdır. Global rubric/cache yöneticisi yalnızca bir tane olmalıdır.

- Q5.1 Her requirement için uygun creational patterni seçin: Abstract Factory, Factory Method, Builder, Prototype, Singleton.
- Q5.2 Her pattern için 1 cümle gerekçe yazın.
- Q5.3 Java benzeri kısa bir sınıf taslağı verin: ExamFamilyFactory, QuestionCreator, ReportBuilder, ExamTemplatePrototype, RubricCache.
- Q5.4 Bu tasarımın OCP ve DIP ile ilişkisini açıklayın.

Soru 6 - Structural GoF pattern seçimi ve UML/snippet

Bir IoT dashboard sistemi geliştiriyorsunuz. Dashboard içinde tekil Widget nesneleri ve iç içe Group nesneleri render edilebilir. Kullanıcı export alırken runtime sırasında Watermark, Compression ve Encryption seçeneklerini zincirleyebilir. Eski bir üçüncü parti ChartLegacy sınıfının draw_raw_chart() metodu vardır; sistem ise tüm görsel öğelerde renderHTML() bekler. Dashboard hem WebClient hem MobileClient hem KioskClient üzerinde çalışır; ayrıca Light, Dark ve Accessible tema implementasyonları bağımsız değişebilmelidir.

- Q6.1 Dört requirement için uygun structural patternleri seçin ve gerekçelerini yazın.
- Q6.2 Metinsel UML çizimi hazırlayın. Stereotype kullanın: <>, <>, <>, <>, <>, <>, <>.
- Q6.3 Kısa client snippet yazın: bir grup oluşturun, legacy chartı adapter ile ekleyin, iki export decorator zincirleyin, MobileClient + DarkTheme ile render edin.
- Q6.4 M x N sınıf patlaması hangi pattern ile önlenir? Neden?

Bölüm 2 - Cevap Anahtarı

Cevap 1

Q1.1 Physical LOC: Boş satırlar sayılmaz, braces sayılır. Kodda 21 non-blank satır vardır. LOC = 21.

Q1.2 Logical LOC: Statements: total=0; for; if; total=...; total=...; return total; result=0; for; result=...; return result. LLOC = 10.

Q1.3 CC(calculateTotal): karar noktaları: for + if = 2. CC = 2 + 1 = 3.

Q1.4 Nesting: for içinde if vardır. Maksimum nesting = 2.

Q1.5 Clone: calculateTotal ve calculateSum tamamen aynı değildir; ikisi de toplama döngüsü yapar ama calculateTotal içinde indirim koşulu vardır. Bu, küçük ekleme/değişiklik ile benzer mantık taşıdığı için Type-3 clone olarak yorumlanabilir.

Q1.6 Halstead: $n_1=8, n_2=5, N=n_1+n_2=14+16=30, n=n_1+n_2=13. V = 30 * \log_2(13) \approx 30 * 3.700 = 111.0$.

Cevap 2

Q2.1 WMC: PremiumSavingsAccount kendi içinde yalnızca applyCashback metodunu tanımlar. Verilen puan 2 olduğundan WMC = 2. Kalıtımla gelen metotlar bu soruda sayılmadı.

Q2.2 DIT: PremiumSavingsAccount -> SavingsAccount -> Account. Account kök sayılırsa DIT = 3.

Q2.3 NOC(Account): Account sınıfının doğrudan çocukları SavingsAccount ve LoanAccount. NOC = 2. PremiumSavingsAccount doğrudan çocuk değildir.

Q2.4 CBO(CustomerProfile): Doğrudan kullandığı sınıflar: Account, EmailSender. String gibi standart tipleri saymazsak CBO = 2.

Q2.5 RFC(CustomerProfile): Kendi metotları: updateName, updateEmail, sendWelcomeMail, printBalance = 4. Doğrudan çağrılan harici metotlar: sender.send, System.out.println = 2. Benzersiz toplam RFC ≈ 6 . Ders/quiz mantığında harici library çağrılarını dahil etmezseniz RFC = 5 (sender.send dahil). Önemli olan mantığı açıklamaktır.

Q2.6 LCOM: 4 metot vardır, toplam çift sayısı $C(4,2)=6$. updateName name alanını, updateEmail ve sendWelcomeMail email alanını, printBalance account alanını kullanır. Ortak alan kullanan tek çift: updateEmail-sendWelcomeMail. $P=1, Q=5. LCOM=\max(5-1,0)=4$. Bu yüksek LCOM düşük cohesion gösterir.

Cevap 3

- Code smells: Magic Numbers (9.99, 19.99, 49.99, 0.80), Switch Statements/Type Conditionals, Long Method (chargeUsers), Large Class (charge, export, email), Duplicated/implicit pricing logic, Shotgun Surgery (yeni plan her yerde değişiklik ister), SRP ihlali.
- Refactoring: Extract Class (PricingService, NotificationService, UserExporter), Extract Method, Replace Conditional with Polymorphism/Strategy, Introduce Interface, Dependency Injection, Rename Constant/Replace Magic Number with Constant.
- Pattern: Plan fiyatlandırma ve indirim davranışı için Strategy uygundur. Her plan veya öğrenci indirimi ayrı strategy olabilir; SubscriptionManager yalnızca strategy kullanır.
- SRP/OCF etkisi: SubscriptionManager kullanıcı listesini yönetir; fiyat, email, export ayrı sınıflara çıkar. Yeni plan eklemek için mevcut sınıfı değiştirmek yerine yeni PricingStrategy eklenir.

```
interface PricingStrategy { double priceFor(User user); }
class BasicPricing implements PricingStrategy {
    public double priceFor(User u) { return u.isStudent() ? 9.99 * 0.80 : 9.99; }
}
class ProPricing implements PricingStrategy {
    public double priceFor(User u) { return u.isStudent() ? 19.99 * 0.80 : 19.99; }
}
class SubscriptionManager {
    private PricingStrategy pricing;
    private NotificationService notifier;
    public SubscriptionManager(PricingStrategy pricing, NotificationService notifier) {
        this.pricing = pricing;
        this.notifier = notifier;
    }
    public void charge(User u) {
        double price = pricing.priceFor(u);
        notifier.notify(u, "Charging: " + price);
    }
}
```

Cevap 4

- SRP: DeliveryOrderProcessor ödeme, indirim, persistence ve bildirim işlerini birlikte yapıyor.
- OCP: paymentType if/else ve gold discount if bloğu yeni ödeme/indirim eklendiğinde mevcut sınıfı değiştirmeyi gerektirir.
- DIP: new CardPayment(), new MySQLOrderRepository(), new EmailNotifier() somut sınıflara bağımlılık yaratır.
- LSP: DroneCourier, Courier yerine her zaman kullanılamaz; belirli ağırlıkta exception fırlatır. Alt sınıf üst sınıfın beklenen davranışını bozuyor.
- ISP: PasswordOnlyLogin, kullanmadığı OTP ve FaceID metotlarını uygulamaya zorlanıyor.

Feature abstractionları: Apple Pay için PaymentGateway; SMS için NotificationService; dinamik kampanya için DiscountStrategy/DiscountPolicy; MongoDB için OrderRepository.

LSP düzeltmesi: Courier hiyerarşisini yeteneklere göre ayırın veya canDeliver(order) sözleşmesi ekleyip processor uygun courier seçsin. Drone, genel Courier yerine WeightLimitedCourier gibi açık contract taşıyan interface uygulayabilir.

ISP düzeltmesi: PasswordLogin, OTPLogin, FaceIdLogin gibi küçük arayüzlere bölün.

```
class DeliveryOrderProcessor {
    private final PaymentGateway payment;
    private final OrderRepository repository;
    private final NotificationService notifier;
    private final DiscountPolicy discounts;

    DeliveryOrderProcessor(PaymentGateway payment,
                           OrderRepository repository,
                           NotificationService notifier,
                           DiscountPolicy discounts) {
        this.payment = payment;
        this.repository = repository;
        this.notifier = notifier;
        this.discounts = discounts;
    }

    public void process(Order order) {
        double total = discounts.apply(order, order.calculateTotal());
        payment.pay(order.paymentType(), total);
        repository.save(order);
        notifier.notify(order.customer(), "Order accepted");
    }
}
```

Cevap 5

- Abstract Factory: Üniversite/Kurumsal/Bootcamp gibi aileler uyumlu Question, GradingPolicy ve ReportFormat nesneleri üretir.
- Factory Method: QuestionCreator alt sınıfları hangi concrete soru tipinin oluşturulacağına karar verir.
- Builder: Report birçok opsiyonel bölümle adım adım kurulur.
- Prototype: Pahalı hazırlanmış ExamTemplate klonlanarak hızlı kopyalanır.
- Singleton: RubricCache/GlobalRubricManager tek instance olmalıdır.

```
interface ExamFamilyFactory {
    Question createQuestion();
    GradingPolicy createPolicy();
    ReportFormat createReportFormat();
}

class UniversityExamFactory implements ExamFamilyFactory { /* compatible family */ }
class CorporateExamFactory implements ExamFamilyFactory { /* compatible family */ }

abstract class QuestionCreator {
    abstract Question createQuestion();
    public Question prepareQuestion() { return createQuestion(); }
}

class CodingQuestionCreator extends QuestionCreator {
    Question createQuestion() { return new CodingQuestion(); }
}

class Report { boolean summary; boolean rubric; boolean analytics; }
class ReportBuilder {
    private Report r = new Report();
    ReportBuilder withSummary(){ r.summary = true; return this; }
    ReportBuilder withRubric(){ r.rubric = true; return this; }
    ReportBuilder withAnalytics(){ r.analytics = true; return this; }
    Report build(){ return r; }
}

interface Prototype<T> { T clone(); }
```

```

class ExamTemplate implements Prototype<ExamTemplate> {
    public ExamTemplate clone() { return new ExamTemplate(/* copied state */); }
}

class RubricCache {
    private static RubricCache instance;
    private RubricCache() {}
    public static synchronized RubricCache getInstance() {
        if (instance == null) instance = new RubricCache();
        return instance;
    }
}

```

OCP/DIP ilişkisi: İstemci concrete sınıfları bilmeden interface/abstract factory üzerinden çalışır. Yeni aile, soru tipi veya rapor varyasyonu mevcut client kodunu değiştirmeden eklenebilir.

Cevap 6

- Composite: Widget ve Group aynı renderHTML arayüzünü uygular; tekil ve bütün aynı şekilde işlenir.
- Decorator: Watermark, Compression, Encryption runtime sırasında zincirlenir; her kombinasyon için subclass gerekmez.
- Adapter: ChartLegacy.draw_raw_chart(), IWidget.renderHTML() arayüzüne çevrilir.
- Bridge: ViewerClient hiyerarşisi ile Theme implementasyonları bağımsız değişir; Web/Mobile/Kiosk x Light/Dark/Accessible için M x N class patlaması önlenir.

```

IWidget <<Component>>
+ renderHTML()
TextWidget <<Leaf>> implements IWidget
ImageWidget <<Leaf>> implements IWidget
Group <<Composite>> implements IWidget
- children: List<IWidget>
+ add(IWidget)
+ renderHTML()

Exporter <<Component>>
BaseExporter <<ConcreteComponent>>
ExportDecorator <<Decorator>> wraps Exporter
WatermarkDecorator <<ConcreteDecorator>>
CompressionDecorator <<ConcreteDecorator>>
EncryptionDecorator <<ConcreteDecorator>>

ChartLegacy
ChartAdapter <<Adapter>> implements IWidget
- legacy: ChartLegacy
+ renderHTML() calls legacy.draw_raw_chart()

ViewerClient <<Abstraction>>
- theme: Theme
WebClient / MobileClient / KioskClient <<RefinedAbstraction>>
Theme <<Implementor>>
LightTheme / DarkTheme / AccessibleTheme <<ConcreteImplementor>>

IWidget text = new TextWidget("CPU usage");
IWidget image = new ImageWidget("diagram.png");
IWidget chart = new ChartAdapter(new ChartLegacy());

Group root = new Group();
root.add(text);
root.add(image);
root.add(chart);
root.renderHTML();

Exporter exporter = new BaseExporter(root);
exporter = new WatermarkDecorator(exporter);
exporter = new EncryptionDecorator(exporter);
String out = exporter.export();

Theme theme = new DarkTheme();
ViewerClient client = new MobileClient(theme);
client.render(root);

```

Kısa Kontrol Listesi - Sınavda Hatırlanacaklar

- LOC: fiziksel satır. LLOC: gerçek statement sayısı.
- Cyclomatic Complexity: karar noktaları + 1. for, if, else-if, while, case genelde karar noktasıdır.
- Nesting: iç içe blokların maksimum derinliği.
- Type-2 clone: aynı yapı/mantık, farklı isimler. Type-3 clone: küçük ekleme/silme/değişiklik var.
- WMC yüksekse class daha karmaşık ve test/maintain zor.
- DIT yüksekse reuse artabilir ama davranışı anlamak ve test etmek zorlaşır.
- NOC yüksekse base class değişiklikleri çok çocuğu etkiler.
- CBO sayısal olarak aynı kalsa bile concrete yerine interface kullanmak coupling kalitesini iyileştirir.
- RFC yüksekse bir mesaja cevap olarak çok metod çalışabilir; test ve hata riski artar.
- LCOM yüksekse cohesion düşüktür; Extract Class/SRP düşün.
- SOLID ipucu: "and" ile açıklanan sınıf sorumluluğu genellikle SRP ihlalidir.
- Switch/type conditional çoğu zaman Strategy, Factory Method veya polymorphism ihtiyacıdır.
- Creational: Abstract Factory = aile; Factory Method = subclass karar verir; Builder = adım adım karmaşık nesne; Prototype = clone; Singleton = tek instance.
- Structural: Composite = parça-bütün; Decorator = runtime ek sorumluluk; Adapter = uyumsuz arayüzü uydurur; Bridge = abstraction ve implementation ayrı değişir.

Kullanılan Quiz Kaynakları

- quiz-2-key.pdf: SOLID ihlalleri, SRP/OCP/DIP/LSP/ISP düzeltmeleri.
- quiz-3-key.pdf: e-commerce feature talepleri, OCP/SRP/DIP refactoring yaklaşımı.
- quiz-4-key.pdf: code smells ve ShoppingCart refactoring örneği.
- quiz-5-key.pdf: LOC, LLOC, Halstead, Cyclomatic Complexity, nesting, clone türleri.
- quiz-6-key.pdf: CK metrikleri - WMC, DIT, NOC, CBO, RFC, LCOM.
- quiz-7-key.pdf: creational pattern seçimleri ve entegre örnek.
- quiz-8-key.pdf: structural pattern seçimleri, Composite/Decorator/Adapter/Bridge.